

3장 파이프라인을 어떻게 나누고, 무엇을 먼저 세울까

3주차 · 파이프라인 아키텍처

1장에서 데이터가 모델까지 가는 길을 보고 2장에서 그 길에 흐르는 데이터가 어떻게 망가지는지 봤다면, 이 장은 그 길 자체를 설계한다. 무엇을 언제 계산할지, 파이프라인을 어떤 역할로 나눌지, 그리고 그 구성을 어떻게 파일 한 장으로 남길지를 정한다.

이 장의 실습은 실제 `docker-compose.yml` 을 파싱해 만든 구성 계획이며, 뒤 장에서 하나씩 세울 계층의 밑그림에 해당한다.

1. 오늘의 질문

- 실시간으로 보여 주는 숫자와 나중에 확정하는 숫자를 한 시스템에서 어떻게 함께 낼까?
- 로직을 고쳤을 때 과거 데이터를 다시 계산하는 일은 누가 감당할까?
- 파이프라인을 계층으로 나눈다는 말은 도구 목록을 나눈다는 뜻일까?
- 구성 파일 한 장이 왜 공공 검수에서 증거가 될까?

2. 건축 현장으로 보는 이 장

이 장의 네 주제는 건물을 짓는 현장에 하나씩 대응한다. 사람이 머릿속으로 아는 것을 도면에 적어 두어야 다른 사람이 같은 것을 다시 만들 수 있다는 점이 같다.

건축 현장	데이터 파이프라인
하자 보수를 전담 팀에 따로 맡길지, 같은 공정으로 처음부터 다시 시공할지 정한다	재처리를 배치 레이어에 맡길지, 로그를 되감는 리플레이로 감당할지 정한다
터파기·골조·설비·마감을 공정으로 나눈다	수집·저장·처리·서빙·모니터링을 계층으로 나눈다
자재 반입 기록이 남아 있어야 하자의 원인을 되짚을 수 있다	원천 로그가 남아 있어야 집계 결과를 되짚을 수 있다
도면 한 장에 자재·치수·순서를 적어 두면 다른 사람이 같은 건물을 세운다	구성 파일 한 장에 서비스·포트·의존을 적어 두면 다른 사람이 같은 환경을 세운다

비유는 여기까지다. 건물은 완공되면 멈추지만 파이프라인은 멈추지 않고 돌며, 도면과 실물이 다르면 눈으로 보이지만 구성 파일과 실제 동작이 어긋나면 실행해 보기 전에는 드러나지 않는다. 이 마지막 차이가 7절 실습의 관찰거리다.

3. 재처리를 어디서 감당할 것인가

파이프라인 설계에서 가장 먼저 정하는 것은 "데이터를 언제 계산하는가"다. 선택지는 세 가지다.

배치(batch)는 데이터를 일정 기간 모았다가 한꺼번에 처리한다. 하루분 전체를 대상으로 계산하므로 늦게 도착한 이벤트까지 포함한 확정 수치를 낼 수 있고, 같은 입력으로 다시 돌리면 같은 결과가 나온다. 대신 지연이 크다. 어제까지의 데이터로 오늘 아침 보고서가 나오는 구조에서는 "지금 무슨 일이 벌어지는가"에 답할 수 없다.

스트리밍(streaming)은 이벤트가 도착하는 즉시 처리한다. 재난 신고가 몰리는 순간의 지역별 건수를 초 단위로 갱신할 수 있다. 대신 집계가 어렵다. 23시 58분에 발생한 민원이 00시 03분에 도착하면 이미 닫힌 집계에 넣을 것인지를 매번 결정해야 한다.

공공 업무는 이 둘을 동시에 요구한다. 재난 상황실은 지금의 신고 건수를 봐야 하고, 사후 감사와 통계 보고는 그날 확정된 정확한 건수를 요구한다. 하나의 처리 방식으로 두 요구를 함께 만족시킬 수 없다는 것이 **하이브리드(hybrid)**의 출발점이며, 이를 구성하는 대표 설계가 Lambda와 Kappa다.

Lambda — 두 경로를 나란히 두고 로직을 두 벌 유지한다

Lambda는 같은 데이터를 두 경로로 동시에 흘려보낸다. 배치 레이어는 원천 전체를 주기적으로 재계산해 확정치를 만들고, 스피드 레이어는 방금 도착한 데이터만 처리해 근사치를 만든다. 조회할 때 둘을 합쳐 "확정된 과거 + 근사한 최근"을 보여 준다. 재난 대시보드로 옮기면 어제까지는 배치의 확정 수치로, 오늘 방금까지는 스피드 레이어의 근사치로 채우고, 자정이 지나면 오늘 분량도 배치가 다시 계산해 확정으로 바꾼다.

정확성에는 대가가 따른다. **같은 집계 로직을 배치용과 스트리밍용 두 벌로 유지해야 한다.** 결국 처리 규칙이나 창 경계 같은 기준 하나를 배치 쪽만 고치고 스트리밍 쪽을 빠뜨리면 같은 시각의 확정치와 근사치가 어긋나는데, 시스템은 멈추지 않는다. 1장의 침묵 실패가 아키텍처 수준에서 다시 나타나는 자리다.

Lambda의 두 레이어는 이 책에서 각각 실물로 확인된다. 스피드 레이어는 6장의 스트리밍 실습이고, 배치 레이어는 7장의 배치 정제 실습이다. 7장에서는 사흘분 민원 60건을 정제하며 총계 보존식 $60 = \text{매핑 } 54 + \text{미기재 } 2 + \text{미매핑 } 3 + \text{중복 } 1$ 을 검증하고, 같은 날짜를 다시 돌리면 산출물 해시가 똑같이 재현된다.

Kappa — 경로를 하나로 두고 로그를 되감는다

Kappa는 배치 레이어를 두지 않고 스트리밍 하나만 유지한다. 재처리가 필요하면 로그에 보관된 과거 이벤트를 처음부터 다시 흘러보내는데, 이것을 **리플레이(replay)**라 한다. 로직이 한 벌뿐이므로 배치와 스트리밍이 어긋날 여지가 구조적으로 사라진다.

성립하는 전제는 이벤트 로그를 보존하고 다시 읽을 수 있어야 한다는 것이다. Kafka 같은 로그 기반 수집 계층은 각 이벤트에 **오프셋(offset)**, 즉 로그 안에서의 위치 번호를 붙이고 원본을 일정 기간 그대로 보관하므로, **컨슈머(consumer)**를 과거 오프셋으로 되돌리면 그 시점부터 다시 읽을 수 있다.

한계도 분명하다. 대규모 과거 데이터 전체를 다시 흘리는 것은 최적화된 배치 스캔보다 비효율적일 수 있고, 복잡한 다차원 집계나 대규모 조인은 여전히 배치 엔진이 유리하다.

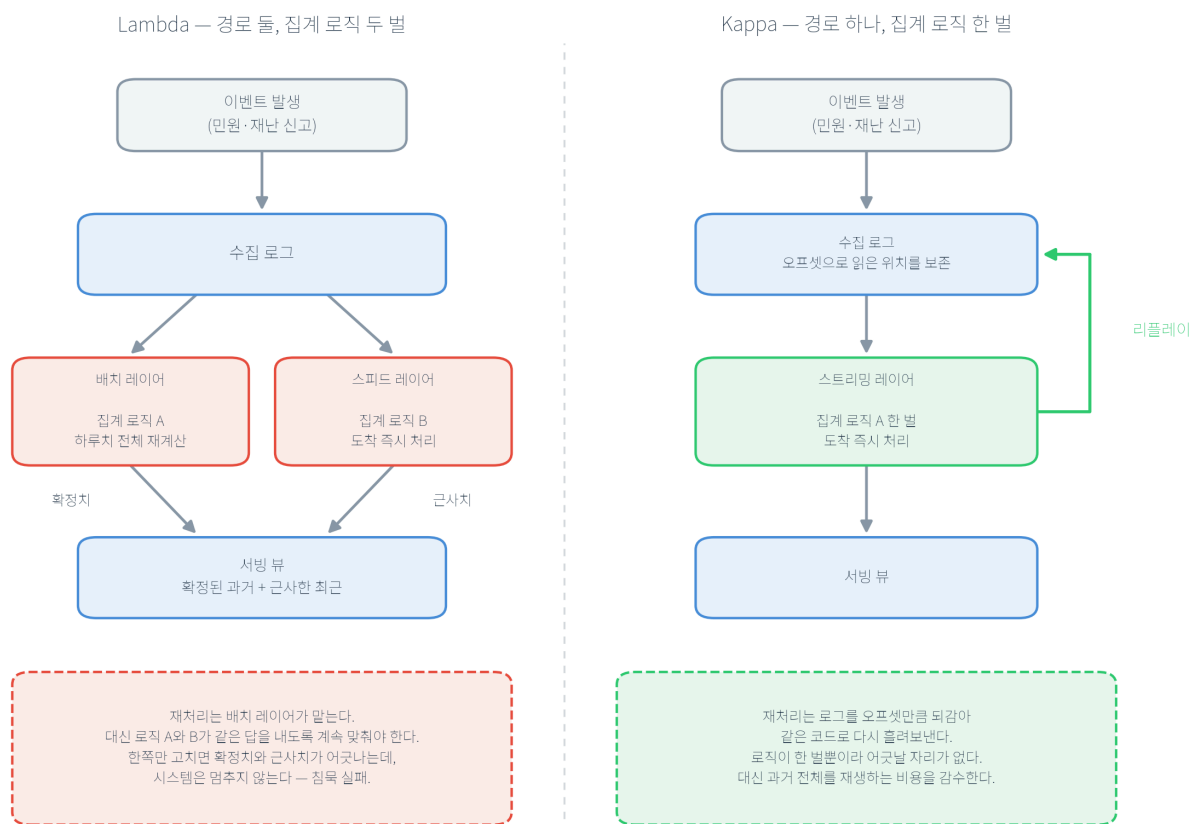


그림 3.1 Lambda와 Kappa: 재처리를 어디서 감당하는가

두 청사진은 위쪽 절반이 같다. 같은 이벤트가 같은 수집 로그로 들어간다. 갈라지는 곳은 그 아래이며, Lambda는 경로가 둘이고 Kappa는 하나다. 오른쪽 초록 화살표가 리플레이로, 로그를 되감아 같은 코드로 다시 흘리는 경로다. **선택의 축은 정확성이 아니라 재처리를 로직 이중화로 감당할 것인가 리플레이 비용으로 감당할 것인가다.** 기술의 우열이 아니라 무엇을 감수할지의 선택이다.

4. 계층은 도구 목록이 아니라 "없으면 무엇이 실패하는가"다

파이프라인을 다섯 계층으로 나눌 때 흔한 오해는 계층을 제품 이름의 서랍으로 보는 것이다. 계층은 역할 분해이고, 그 역할은 **그 계층이 없을 때 무엇이 실패하는가**로 정의된다.

수집 계층은 생산과 소비의 속도 차이를 흡수한다. 재난 문자 발송 직후 민원이 초당 수백 건 들어올 때 하류 서버는 그 속도를 못 따라가는데, 생산자와 소비자를 직접 이으면 속도 차이가 곧 장애가 된다. 완충 로그를 두면 빠르게 받고 천천히 처리할 수 있다. 이 계층의 핵심 판단은 전달 보증 수준의 선택이다. 4장에서 커밋 시점 한 줄만 바꾸면 at-most-once는 **크래시(crash)** 시 20건을 오류 로그 없이 유실하고, at-least-once는 10건을 중복 수신한다.

저장 계층은 원천과 가공 데이터를 보존하고 접근을 통제한다. 원천을 버리면 세 가지가 불가능해진다. 변환 로직 버그가 나중에 발견됐을 때의 재처리, "그때 받은 그대로"를 요구받는 감사, 처음에는 안 쓰던 필드가 나중에 필요해지는 새 요구다. 셋 다 요약본으로는 복원되지 않는다.

처리 계층은 수집된 데이터를 변환·집계한다. 원천 이벤트는 그대로는 지표가 되지 못하므로 지역명을 표준 코드로 바꾸고 시간 창으로 묶어 센다. 여기서 늦게 도착한 이벤트를 얼마나 기다렸다 창을 닫을지 정하는 것이 2장에서 본 **워터마크(watermark)** 이고, 6장에서 직접 실행한다.

서빙 계층은 조회·리포팅·예측 요구에 통제된 성능으로 데이터를 내준다. 운영에서의 질문은 "새 모델을 어떻게 넣는가"보다 **"잘못됐을 때 어떻게 되돌리는가"**다. 10장의 새도 배포 실습에서 이용자에게는 champion 모델의 예측 6.0이 나가고 로그에는 challenger 모델의 예측 7.3636이 조용히 쌓여, 이용자 위험 없이 후보를 재평가할 자료가 된다.

모니터링 계층이 없으면 침묵 실패가 발견되지 않는다. 무엇을 볼지의 출발점은 Google SRE의 골든 시그널 네 가지, 즉 지연·트래픽·오류·포화다. 11장은 PSI 0.3324가 경보 대역인데 KS 검정은 유의하지 않은 상태를 실측해 단일 지표로 판정하면 안 되는 이유를 보인다. 단일 워커의 CPU가 106.3%로 포화하는 지점을 재는 방법은 실무교재 `docs/chapter12.md`에 있다.

표 3.2 5계층과 없을 때 실패하는 것

계층	없으면 실패하는 것	대표 도구	이 교재의 실측
수집	유실·과부하	Kafka	4장(유실 20건·중복 10건)
저장	재처리·감사 불가	Postgres	5장, 13장
처리	원천이 지표가 되지 못함	Spark, Airflow	6장, 7장
서빙	데이터가 서비스되지 않음	FastAPI	10장(6.0 / 7.3636)
모니터링	침묵 실패 미발견	Prometheus, Grafana	11장(PSI 0.3324), 실무교재 §12.3(CPU 106.3%)

한 가지를 미리 못 봐야 둔다. **계층과 서비스는 1대 1이 아니다**. 수집 계층 하나를 세우는 데 서비스 둘이 들고 모니터링 하나에 셋이 드는 일이 실제로 일어나는데, 7절 실습에서 이 어긋남을 손으로 확인한다.

5. 리플레이는 공공에서 감사 대응 능력이 된다

민간에서 리플레이는 주로 버그를 고친 뒤 다시 계산하는 수단이다. 공공에서는 여기에 하나가 더 붙는다. 감사관이 "왜 이 통계가 이렇게 나왔는가"를 물으면, 답은 결국 "**그 시점에 들어온 그 데이터로 그 계산을 다시 만들어낼 수 있는가**"로 좁혀진다. 로그를 보존하고 오프셋으로 되감을 수 있는 아키텍처는 재현해 보이겠다고 답할 수 있고, 요약본만 남긴 아키텍처는 답할 수 없다.

그래서 공공 설계에서 지켜야 할 원칙은 하나다. **증거를 사람이 사후에 쓰는 문서가 아니라 시스템이 실행 중에 남기는 산출물로 둔다**. 9장은 모델 훈련 입력의 지문(sha256)을 기록해 같은 데이터·설정이면 같은 모델이 나오는지 재실행으로 검증한다. 13장은 역할과 목적을 함께 보는 접근 정책으로 요청 12건을 판정해 허용 8건·거부 4건을 로그에 남기고, 그 로그에서 10개 항목의 감사 점검표를 자동으로 만든다. "접근 통제가 있다"는 말의 운영적 의미는 이 로그와 점검표를 제시할 수 있다는 뜻이다.

같은 이유로 장애 대응도 문서로 확인되지 않는다. 재시도·먹통 처리·격리가 실제로 동작하는지는 고장을 일부러 내 보기 전에는 확인되지 않으며, 14장에서 오염 이벤트 격리·중복 재전송 흡수·부분 실패 격리 세 가지를 드릴로 돌린다.

Lambda와 Kappa 중 무엇을 고를지는 이 지점에서 수치로 좁혀진다. 하나는 **재처리 소요 시간**이다. Lambda는 배치 잡을 다시 돌리는 시간으로, Kappa는 보관된 로그를 되감아 재생하는 시간으로 잔다. 로직을 고친 뒤 확정치를 며칠 안에 다시 내야 하는지가 정해져 있으면 그 기한이 두 방식의 상한이 된다. 다른 하나는 **종단 지연(end-to-end latency)**으로, 이벤트 발생부터 서빙까지 걸리는 시간이다. 민간은 실시간 추천 100ms 이하, 광고 입찰 10ms 이하를 목표로 잡는 경우가 많지만, 공공에서 이 값의 기준은 사업 성과가 아니라 **대응 시한**이다. 상황실이 몇 초 안에 갱신된 건수를 봐야 하는가가 곧 스트리밍 계층의 요구가 된다.

6. 구성 파일 한 장이 설계도이자 검수 대상이다

앞의 세 절이 무엇을 세울지의 판단이라면, 이 절은 그 판단을 어떻게 파일로 남기는지를 다룬다.

Docker는 애플리케이션과 실행에 필요한 파일·라이브러리·설정을 하나로 묶어 **컨테이너(container)**라는 격리된 공간에서 돌리는 도구다. 여기서 격리는 컴퓨터 안에 또 다른 완전한 컴퓨터를 만든다는 뜻이 아니라, 호스트의 운영체제 자원을 함께 쓰면서 프로세스·네트워크·파일 공간만 서로 구분한다는 뜻이다.

`docker run`이 컨테이너 하나를 실행하는 명령이라면, `docker compose`는 여러 컨테이너와 그 연결 관계를 한 번에 관리하는 명령이다.

Compose 파일은 **선언적 구성**이다. 명령을 순서대로 입력하는 절차적 방식과 달리 "무엇이 있어야 하는가"의 최종 상태를 적으므로, 구성 자체가 사람이 읽는 문서이면서 실행 가능한 설계도가 된다. 이 성질이 공공 검수에서 특히 중요하다. 발주자가 같은 파일로 같은 구성을 다시 세워 확인할 수 있기 때문이다.

무엇을 세우고 무엇을 미룰 것인가

최소 구성의 기준은 하나다. **이 장의 개념을 눈으로 확인할 수 있는가.** 3장에서는 수집·저장·모니터링을 세우고 처리와 오케스트레이션은 각각 6·7장으로 미룬다. 이것은 생략이 아니라 범위 조정에 따른 설계 판단이며, 세 질문으로 정한다.

1. 이 계층을 빼면 이 장에서 보이려는 것이 관찰되지 않는가. 수집·저장·모니터링을 빼면 "계층이 나뉘어 있다"는 사실 자체가 보이지 않는다.
2. 이 계층이 없어도 뒤 실습이 막히지 않는가. 처리 계층은 6장에서 처음 필요하므로 3~5장은 진행된다.
3. 올렸을 때 드는 학습 부하가 그 계층에서 얻는 관찰보다 큰가. Spark와 Airflow는 설정 항목이 많아 이 장에서는 부하가 관찰을 넘어선다.

세 질문을 뒤집으면 추가 시점도 정해진다. **실습이 막히는 시점이 그 계층을 더할 시점이다.**

비밀은 구성 파일에 두지 않는다

선언적 구성에는 함정이 하나 붙는다. 구성이 문서가 된다는 말은 그 파일이 저장소에 커밋되어 여러 사람이 본다는 뜻이기도 하다. 그래서 비밀번호와 인증키는 코드나 구성 파일이 아니라 환경변수·시크릿 저장소에 두고, 파일에는 참조만 남긴다.

이 원칙을 판정할 때 값이 얼마나 복잡한지는 기준이 아니다. 7절 실습의 `compose` 파일에서 `POSTGRES_PASSWORD` 는 `secure_password` 인데도 경고가 뜬다. 경고가 가리키는 것은 값의 복잡도가 아니라 **평문이 파일에 존재한다는 사실 자체다.**

7. 실습 3.1 Docker Compose 구성 추출과 대조

실습 목표

요구사항과 5계층 모델에서 필요한 서비스·포트·의존성을 먼저 스스로 정해 적고, `docker-compose.yml` 에서 같은 항목을 추출한 산출물과 대조한다. 남이 만든 파일을 먼저 열면 거기 적힌 목록이 정답처럼 보여 "왜 이 여섯 개인가"를 묻지 않게 되므로, 순서를 설계 → 실행 → 대조로 고정한다.

1단계 설계 — 구성 계획을 먼저 적는다

가상 발주 조건은 넷이다. 민원 이벤트를 실시간으로 받아 저장하고 유입 현황을 대시보드로 본다. 감사에 대비해 원천 이벤트를 일정 기간 되감아 다시 읽을 수 있어야 한다. 스트리밍 집계와 배치 정제는 이 단계에서 구현하지 않는다(각각 6·7장). 노트북 한 대에서 전부 돌아가야 한다.

표 3.4 로컬 구성 설계표 — `compose` 파일을 열기 **전에** 채운다

계층	넣는가(예/미름)	내가 고른 서비스	열 호스트 포트	무엇에 의존하는가	근거
수집					
저장					
처리					
서빙					
모니터링					

근거 칸에 "기본값이라서"를 쓰지 않는다. 요구사항의 어느 문장에서 나왔는지, 또는 4절의 "없으면 실패하는 것" 중 무엇인지를 가리킨다. "미름"으로 적은 계층에는 언제 추가할지를 6절의 세 질문으로 함께 적는다. 표를 다 채우면 실행 전에 서비스 수·호스트 포트 수·의존성 수 세 숫자를 예측값으로 적어 둔다.

2단계 실행

```
cd practice/chapter3
python3 code/3-1-compose-plan.py
```

핵심 코드

```
data = yaml.safe_load(COMPOSE_PATH.read_text(encoding="utf-8"))
services = data.get("services", {})
OUTPUT_PATH.write_text(json.dumps(plan, ensure_ascii=False, indent=2) + "\n")
```

전체 코드는 *practice/chapter3/code/3-1-compose-plan.py* 참고

실행 결과 확인 사항

산출물은 `practice/chapter3/data/output/ch3_compose_plan.json` 이다. 다음 다섯 항목을 본인 파일에서 확인한다.

1. `compose_path` 와 `generated_at` : 파싱한 compose 파일의 절대 경로와 생성 시각이다. 경로는 각자 노트북의 위치이므로 학생마다 다르고, 이 차이가 본인이 실행해 만들었다는 표시가 된다.
2. 서비스 6개: zookeeper, kafka, postgres, prometheus, grafana, kafka-ui.
3. 호스트 포트 7개: 2181, 3000, 5432, 8080, 9090, 9092, 29092.
4. 추출된 의존성 2건: grafana → prometheus , kafka-ui → kafka .

5. 보안 경고 2건: postgres: weak/default password in POSTGRES_PASSWORD , grafana: weak/default password in GF_SECURITY_ADMIN_PASSWORD .

3단계 대조 — 예측과 산출물 맞춰 보기

예측한 세 숫자 옆에 실제 값을 적고, 어긋난 항목의 원인을 규명한다.

서비스 수를 6보다 적게 예측했다면 계층 하나를 서비스 하나로 셈했기 때문이다. 수집에 kafka와 zookeeper 둘이 들고 모니터링에 prometheus-grafana-kafka-ui 셋이 든다. 4절 끝에서 못 박아 둔 "계층과 서비스는 1대 1이 아니다"가 여기서 숫자로 나타난다.

처리·서빙을 "예"로 적었다면 실제 구성에는 그 자리가 비어 있다. 처리와 오케스트레이션은 6·7장, 서빙 API는 10장에서 붙는다. 요구사항 셋째 줄이 그 판단의 근거였다.

포트가 서비스보다 하나 많은 이유는 kafka가 9092와 29092 둘을 열기 때문이다. 9092는 호스트에서 접속하는 통로(PLAINTEXT_HOST)이고 29092는 컨테이너끼리 쓰는 통로(PLAINTEXT)다. "서비스 하나에 포트 하나"라는 예측은 여기서 깨진다.

의존성을 3으로 예측했다면 그 예측이 맞다. compose 파일에는 kafka → zookeeper , grafana → prometheus , kafka-ui → kafka 셋이 있는데 산출물에는 둘뿐이다. 스크립트가 리스트 형태로 적힌 depends_on 만 읽고, kafka가 헬스체크 조건(맵 형태)으로 적은 의존은 지나쳤기 때문이다. 여기서 어긋난 것은 설계가 아니라 도구이며, 자동 추출 산출물만 근거로 "의존성은 2개"라고 보고하면 그 보고가 틀린다.

마지막 항목이 이 실습의 핵심이다. 예측과 산출물이 어긋났을 때 자기 판단을 고칠지 도구를 고칠지는 원본과 대조해야 정해진다. 설계를 먼저 적지 않으면 이 질문 자체가 생기지 않고 "의존성 2개"를 그대로 받아 적게 된다.

8. 핵심 요약

- 1. 하이브리드의 선택은 재처리를 무엇으로 감당하느냐다.** Lambda는 배치와 스피드 두 경로로 확정치와 근사치를 함께 내지만 같은 집계 로직을 두 별 유지해야 하고, 한쪽만 고치면 두 수치가 어긋난 채 시스템은 멈추지 않는다. Kappa는 로직 한 벌에 리플레이로 재처리를 감당하는 대신 대규모 과거 재생 비용을 진다.
- 2. 계층은 도구 목록이 아니라 없을 때 실패하는 것의 목록이다.** 수집이 없으면 유실·과부하가, 저장이 없으면 재처리와 감사가, 처리가 없으면 지표가, 서빙이 없으면 서비스가, 모니터링이 없으면 침묵 실패 발견이 무너진다. 4장의 유실 20건·중복 10건이 수집 계층을 뺐을 때 무슨 일이 벌어지는지의 실측이다.
- 3. 공공에서 리플레이는 감사 대응 능력이다.** "왜 이 통계가 이렇게 나왔는가"에 답하려면 그 시점의 데이터로 그 계산을 다시 만들 수 있어야 한다. 그래서 증거는 사람이 사후에 쓰는 문서가 아니라 시스템이 실행 중에 남기는 산출물이어야 하며, 13장의 허용 8건·거부 4건 로그와 10항목 감사 점검표가 그 형태다.
- 4. 최소 구성은 "이 장의 개념을 눈으로 확인할 수 있는가"로 정한다.** 3장은 수집·저장·모니터링 셋을 세우고 처리·오케스트레이션 둘을 미룬다. 미루는 판단은 관찰 가능성·후속 실습 진행·학습 부하 세 질문으로 내리고, 실습이 막히는 시점이 그 계층을 더할 시점이다.

5. 구성 파일은 설계도이자 검수 대상이며, 평문 비밀은 값이 복잡해도 경고 대상이다. `POSTGRES_PASSWORD` 가 `secure_password` 인데도 경고 2건이 나오는 것은 값의 복잡도가 아니라 평문이 파일에 있다는 사실을 가리키기 때문이다. 동시에 자동 추출 산출물은 원본과 대조해야 한다. 의존성 3건 중 2건만 잡힌 것이 그 이유다.

9. 과제

실습 3.1을 실행해 만든 다음 두 가지를 LMS에 제출한다.

1. 표 3.4 구성 설계표 — 3단계 대조를 반영한 최종본. 텍스트든 이미지든 형식은 무방하다.
2. `practice/chapter3/data/output/ch3_compose_plan.json` — 실행해 생성된 파일 그대로.

이어서 세 가지를 각각 세 문장 이내로 쓴다.

1. 본인 산출물의 `compose_path` 와 `generated_at` 을 그대로 옮겨 적고, `compose_path` 가 학생마다 다른 이유를 쓴다.
2. 산출물에 기록된 서비스 이름 전부와 각 서비스가 여는 호스트 포트를 적고, 예측한 서비스 수와 어긋났다면 그 원인을 쓴다.
3. 보안 경고 항목 두 건을 그대로 옮겨 적고, 이를 검수 가능한 요구사항 한 문장으로 바꿔 쓴다.

값을 지어내지 않는다. 실행이 안 되면 어디서 막혔는지 그대로 쓴다. 수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter3.md` 에 있다.

- 저장 계층의 최신 논의 — 레이크와 웨어하우스를 하나로 합치는 레이크하우스: `docs/chapter3.md` §3.2.2
- 개인정보와 법령 조문 — 목적 외 이용 제한, 민감정보, 영향평가, 자동화된 결정에 대한 권리: §3.3.3 (16장에서 본격적으로 다룬다)
- 장애 대응을 숫자로 — RTO와 가용성 99.9%·99.99%의 뜻: §3.3.4
- 검수 가능한 요구사항 문장 만들기 — 발주 언어로 바꾸는 규칙과 예시: §3.3.5
- 배포 환경과 비용 — 공공 클라우드·망분리·데이터 등급, 측정 후 증설 원칙: §3.4.4~§3.4.5
- 실습을 더 해 보려면 — `compose` 파일에 서비스 추가, `map-form` 의존성까지 읽도록 파서 보강, 포트 충돌 유발: `practice/chapter3/` 및 §실습 3.1 실습 확장

원문을 직접 보고 싶다면 다음 두 편이 출발점이다.

- Pathirage, 「Kappa Architecture」 — Kappa의 정의와 리플레이 전제를 정리한 문서. <https://milinda.pathirage.org/kappa-architecture.com/>
- The Twelve-Factor App, 「III. Config」 — 비밀과 구성을 환경에 두는 원칙. <https://12factor.net/config>